

PRE-CLASS READING 3 OF 5

Software 2.0

Karpathy, A. (2017). "Software 2.0." Medium. Published 11 November 2017.

ASSIGNED READING

Full essay ~12 min

» ~12 min

Note on currency. Written in 2017, before the transformer boom — its examples are ConvNets and AlphaGo, not LLMs. The thesis, that neural-network weights are a new kind of source code written by optimisation over data, is more true in 2026 (foundation models) than when it was written. Read it as the conceptual root of why the modern stack looks the way it does.

KEY TAKEAWAY

Neural networks are not just another classifier — they are a new way to write software. In Software 1.0 a human writes explicit instructions. In Software 2.0 you specify a goal (a dataset plus a loss function) and an optimisation process searches program-space for the weights that satisfy it. The dataset is the source code; training is the compiler; the trained network is the binary.

WHY IT MATTERS FOR THIS COURSE

The 'why' beneath Class 2's 'what.' Karpathy's dividing line is simple: Software 1.0 is logic a human writes by hand (rules, lookups); Software 2.0 is a program whose parameters are learned from data by optimisation. Both things you compared in Class 1 sit on the 2.0 side — your scikit-learn classifier learns its weights from data (a small, mostly-interpretable 2.0), and the foundation-model API is 2.0 at massive scale. True Software 1.0 would be the hand-coded rule you'd write instead. It underpins C1 (the modern stack is a data-and-model pipeline, not hand-written code) and C4 (evaluation exists precisely because 2.0 fails silently). Reading 2 (Huyen) then turns this philosophy into day-to-day engineering.

Feeds Class 2: C1 (map the stack) • C4 (evals as a layer) • sets up Reading 2 (Huyen).

MAIN POINTS

1.0 vs 2.0

- **Software 1.0.** Explicit instructions written by a programmer in Python, C++, etc. — each line pins a specific point in program space.
- **Software 2.0.** The program is the weights of a neural network. Humans specify a goal and a rough architecture skeleton; stochastic gradient descent and backpropagation search a continuous slice of program space for weights that work.
- **Where classical ML sits.** A scikit-learn classifier (e.g. logistic regression) still learns its weights from data, so it is a small, interpretable Software 2.0 — not 1.0. Only hand-coded rules are 1.0. Karpathy does call the older 'hand-engineered features + a bit of ML' pipelines more 1.0-ish, but a model trained from data sits on the 2.0 side.
- **The compile analogy.** In 1.0, source code compiles to a binary. In 2.0, the 'source' is the dataset + architecture, and training compiles the dataset into the network. Architectures and training systems are increasingly commodity — so most real work is curating, growing and cleaning labelled datasets.
- **The teams split.** 2.0 'programmers' edit and grow datasets; a few 1.0 programmers maintain the training and tooling around them. 'AI (Software 2.0) is eating software.'

The transition is already everywhere

- Vision, speech recognition, speech synthesis, translation, and games (AlphaGo Zero) have all been ported from hand-written 1.0 code into learned 2.0 systems — even parts of databases (learned index structures).

Why 2.0 is attractive

- **Homogeneous + hardware-friendly.** Mostly matrix-multiply + ReLU, so it is easy to reason about and easy to bake into silicon (custom chips).
- **Predictable + portable + agile.** Constant run-time and memory; runs on many configurations; trim channels and retrain to run 2x faster at slightly lower quality — tuning that is trivial in 2.0 and painful in 1.0.
- **Modules meld; often 'better than you.'** Two separately trained modules can be back-propagated through as one whole, and learned code beats hand-written code across perception-heavy verticals.

Why 2.0 is dangerous — and why evals exist

- **Opaque.** You may face a choice between a 90%-accurate model you understand and a 99%-accurate model you don't.
- **Fails silently.** It can quietly absorb biases in the training data, and is vulnerable to adversarial examples — failures that are hard to notice or analyse. This is the root reason the stack needs evaluation and observability layers (Reading 4, capsule C4).

Programming in the 2.0 stack

- **You program by curating data.** To fix a rare failure you don't write code — you add labelled examples. Karpathy calls for 2.0 IDEs, a 'Software 2.0 GitHub' (repos are datasets, commits are label edits), and package managers for weights — much of which now exists (model hubs, experiment trackers, registries).

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall.

1. Karpathy says in 2.0 'the dataset is the source code.' If a model in your domain misbehaves, the fix is changing the data, not editing code. What data would you add or remove to fix a specific failure you can picture?
2. He names 'fails silently' and 'hard to understand' as the core weaknesses of 2.0. Where in your industry would a silent, confident error be most dangerous — and how would anyone even notice it happened?
3. In 2017 the examples were ConvNets; today's foundation models are the same idea at massive scale. What, if anything, does the 'Software 2.0' framing get wrong or leave out about modern LLMs?

SOURCE LINKS

Essay · <https://karpathy.medium.com/software-2-0-a64152b37c35>

Companion (optional) · Karpathy, 'Intro to Large Language Models' (Class 1 Reading 3)

CAVEATS

1. This summary is not a substitute for the reading. It is a map, not the territory. Read the source — it is the authority.
2. This summary was generated and fact-checked using Anthropic models against the original 2017 essay. Flag anything that looks wrong.